Apache Airflow

1. What is Apache Airflow?

Apache Airflow is an open-source workflow orchestration platform originally created at Airbnb and later donated to the Apache Software Foundation. It lets you programmatically author, schedule, and monitor data pipelines — all in Python.

Think of Airflow as a smart traffic controller for your data jobs. Instead of running scripts manually or chaining fragile cron jobs, you define the full workflow once and Airflow handles scheduling, dependency management, retries on failure, and centralized logging automatically.

How it works — A simple example

Suppose a company wants a fresh sales report every morning. Without Airflow, someone manually downloads data, cleans it, runs the report, and emails it. With Airflow, this entire chain runs automatically at 6 AM every day:

- Download sales data from the API
- Validate and clean null values
- Load cleaned data into the data warehouse
- Refresh the dashboard
- Send a success or failure email alert

Key Components

Component	What it does
DAG	The complete workflow — a set of tasks wired together
Task	One individual step inside a DAG
Operator	Defines what a task does (Python, Bash, SQL, etc.)
Scheduler	Decides when DAGs run and submits tasks to the executor
Executor	Runs the actual tasks (runs inside the Scheduler process)

Web UI	Browser dashboard to monitor, trigger, and debug DAGs
Metadata DB	Stores task status, run history, logs, and schedule info

What Airflow is used for

- ETL / ELT data pipelines
- Data warehouse loading and transformation
- Machine learning pipeline scheduling
- Report automation and delivery
- Multi-system cloud job orchestration (Spark, Databricks, Snowflake, etc.)

2. Why Airflow Instead of Azure Data Pipelines?

Azure offers its own orchestration tools — primarily Azure Data Factory (ADF) and Databricks Workflows. These are excellent for workloads that live entirely inside the Azure ecosystem. So why would you choose Apache Airflow?

When Azure-only tools are enough

If your entire flow stays within Azure and Databricks — read a CSV from ADLS, transform it with PySpark, write a Delta table, run another notebook, schedule daily — you do NOT need Airflow. Databricks Jobs or Workflows handles this cleanly.

When Airflow becomes the better choice

Airflow shines when you need to orchestrate across many different systems that are not all from one vendor. For example:

1. Check if a file arrived on an SFTP server
2. Call an external REST API
3. Trigger an Azure Databricks notebook
4. Run an Azure Data Factory pipeline
5. Load output into Snowflake
6. Send a Slack or email alert
7. Trigger a Power BI dashboard refresh

No single Azure tool handles all of that well. Airflow can coordinate all of it from a single DAG.

Comparison: Databricks Jobs vs Apache Airflow

Feature	Databricks Jobs	Apache Airflow

Runs Databricks notebooks	Yes (native)	Yes (via operator)
Runs Spark jobs	Yes	Yes (indirectly)
Databricks-only workflows	Excellent fit	Overkill
Multi-system orchestration	Limited	Excellent
Cloud/platform independent	No — Databricks-only	Yes — any system
DAG-style dependencies	Yes	Yes
Retry support	Yes	Yes
UI monitoring	Yes	Yes
Best role	Run Databricks workloads	Orchestrate across platforms

Airflow is not a replacement for Databricks — it is the external conductor. Databricks is where the heavy computation happens. Airflow decides when and in what order everything runs across your entire stack.

3. What is a DAG?

DAG stands for Directed Acyclic Graph. It is the core concept that makes Airflow work.

Breaking it down

- Directed — tasks flow in one direction only, from earlier steps to later steps
- Acyclic — there are no loops or cycles; a task cannot eventually point back to itself
- Graph — tasks are nodes and the arrows between them are the dependencies

In Airflow, a DAG is your entire workflow written as a Python file. Each task inside the DAG represents one unit of work (an extract, a transform, a load, an alert). The arrows between tasks define the order they must run.

Simple DAG example

Here is what a four-step data pipeline DAG looks like as code:

```
from airflow.sdk import DAG
from airflow.providers.standard.operators.python import PythonOperator
from datetime import datetime

def extract():  print('Extracting data...')
def transform(): print('Transforming data...')
def load():     print('Loading to warehouse...')
def notify():   print('Sending email alert...')

with DAG(
    dag_id='sales_pipeline',
    start_date=datetime(2025, 1, 1),
```

```
    schedule='@daily',
    catchup=False
) as dag:

    t1 = PythonOperator(task_id='extract',   python_callable=extract)
    t2 = PythonOperator(task_id='transform', python_callable=transform)
    t3 = PythonOperator(task_id='load',      python_callable=load)
    t4 = PythonOperator(task_id='notify',    python_callable=notify)

    t1 >> t2 >> t3 >> t4   # chain: extract → transform → load → notify
```

The >> operator is how you define dependencies — it means 'run this task after that one'. Airflow reads the DAG file, draws the graph, and executes tasks in the correct order on schedule.

Key DAG properties

Property	What it means
dag_id	Unique name for the DAG
start_date	The earliest date the DAG can run from
schedule	When to run — @daily, @hourly, or a cron expression like '0 6 * * *'
catchup=False	Do not run all missed past runs — only run from now
tags	Labels to organise DAGs in the UI

Every time Airflow runs a DAG, it creates a DAG Run. Inside that run, each task creates a Task Instance. You can monitor all of these from the Web UI in real time.

4. Setting Up Airflow on Killercoda (Play With Docker)

The following steps walk through spinning up Apache Airflow using a single Docker container on the Killercoda Play With Docker lab environment, creating a DAG file manually, and verifying it loads correctly.

Step 1 — Launch Airflow in Docker

Run the official Airflow standalone container, mapping the Web UI to port 8080:

```
docker run -d \
  -p 8080:8080 \
  --name airflow \
  apache/airflow standalone
```

The -d flag runs the container in the background. The standalone image bundles the scheduler, web server, and database into a single container — perfect for learning.

Step 2 — Verify the container is running

```
docker ps
```

You should see the airflow container listed with status Up.

Step 3 — Get the admin password from logs

```
docker logs airflow
```

Scroll through the output and look for a line like:

```
username: admin
password: <auto-generated-password>
```

Copy that password — you will need it to log into the Web UI.

Step 4 — Open the Airflow Web UI

In the Killercoda / Play With Docker toolbar, click OPEN PORT and enter 8080. The Airflow Web UI will open in your browser. Log in with username admin and the password from the logs.

Step 5 — Enter the Airflow container shell

```
docker exec -it airflow bash
```

This drops you into a bash shell inside the running container as the airflow user.

Step 6 — Navigate to the DAGs folder

```
cd /opt/airflow/dags
```

Note: Make sure to put a space between cd and the path — cd/opt/... (without a space) gives a 'No such file or directory' error.

Step 7 — Create the first DAG file

Use the cat > command to write a new Python file. After typing the command, paste the DAG code, then press Ctrl+D to save:

```
cat > first_dag.py
```

Then paste the following DAG code:

```
from airflow.sdk import DAG
from airflow.providers.standard.operators.python import PythonOperator
from datetime import datetime


def task1_function():
    print('Task 1 Started')


def task2_function():
    print('Task 2 Running')


def task3_function():
    print('Task 3 Completed')


with DAG(
    dag_id='first_training_dag',
    start_date=datetime(2025, 1, 1),
    schedule='@daily',
    catchup=False,
    tags=['training']
) as dag:

    task1 = PythonOperator(
        task_id='task1',
        python_callable=task1_function
    )

    task2 = PythonOperator(
        task_id='task2',
        python_callable=task2_function
    )

    task3 = PythonOperator(
        task_id='task3',
        python_callable=task3_function
    )

    task1 >> task2 >> task3
```

Press Ctrl+D to finish writing and save the file.

Step 8 — Verify the file was created

```
ls /opt/airflow
```

You should see first_dag.py listed alongside airflow.cfg, dags/, and logs/.

Step 9 — Check for import errors

```
airflow dags list-import-errors
```

If the output shows 'No data found', your DAG has no syntax errors and loaded successfully.

Step 10 — Force re-serialization and list DAGs

```
airflow dags reserialize
airflow dags list
```

After a few seconds, first_training_dag should appear in the list. If not, wait 30 seconds and run the commands again — the Airflow scheduler picks up new DAG files on a polling interval.

5. Executing Your First DAG

After the DAG file is saved and the scheduler has picked it up, you can trigger and monitor it from the Airflow Web UI.

Step 1 — Open the DAGs list in the Web UI

Navigate to the Airflow Web UI (port 8080). You should see first_training_dag listed with the tag 'training' next to its name.

Step 2 — Enable the DAG

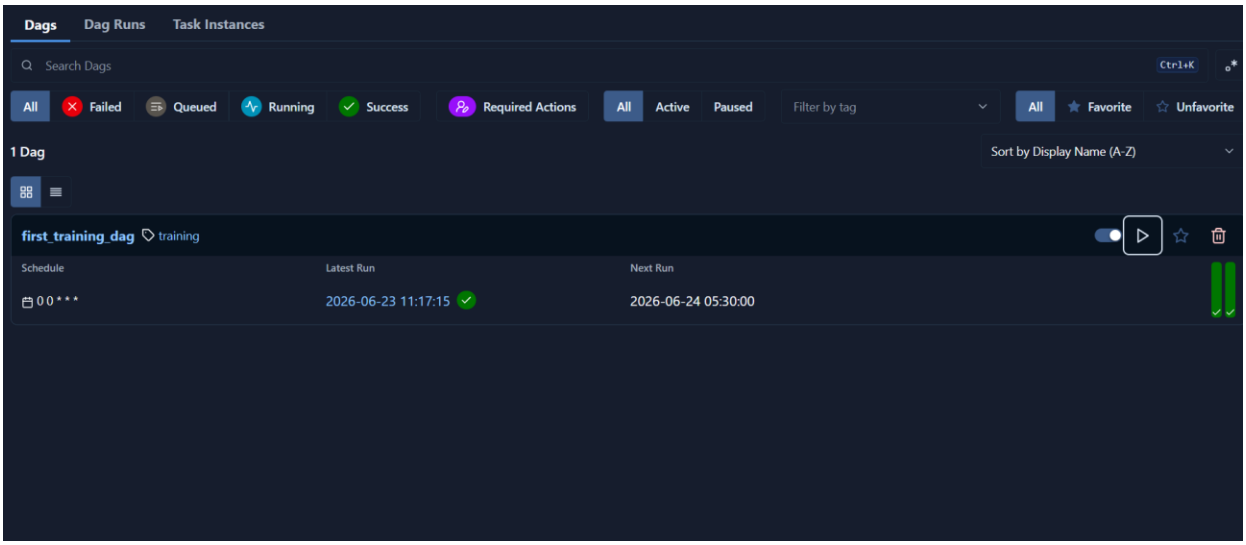
By default, new DAGs are paused. Click the toggle switch on the left of the DAG name to enable it. The toggle turns blue when active.

Step 3 — Trigger the DAG manually

Click the Play button (triangle icon) on the right side of the DAG row, then confirm the trigger. Airflow will immediately create a new DAG Run.

Step 4 — Watch the run succeed

The Latest Run column updates with a timestamp and a green checkmark, confirming all tasks completed successfully. The screenshot below shows this state:



Step 5 — Verify tasks in Graph View

Click on the DAG name to open its detail view. Switch to Graph View to see the three tasks (task1 → task2 → task3) all highlighted in green, confirming every task instance ran and completed without errors.

Step 6 — Check task logs

Click on any task node in the Graph View, then click Logs. You will see the print output from the Python function — Task 1 Started, Task 2 Running, or Task 3 Completed — confirming the function executed correctly inside Airflow.